

A Multi-Source Passive Credential Exposure Scanner with Graceful API Degradation

Ali Murtaza Bhutto
Independent Researcher
ORCID: 0009-0007-2787-943X
Email: alibhutto101112@gmail.com

Abstract—We describe `credential-leak-scanner`, a small Python pipeline that combines three structurally different passive sources — the HaveIBeenPwned (HIBP) Pwned Passwords k-anonymity endpoint, the keyed HIBP breached-account endpoint, and a local synthetic breach corpus — into a single defensive credential-exposure report. The contribution is not novel cryptography; the underlying k-anonymity protocol is well studied [1], [5]. Rather, we report on three practical design choices and their measurable consequences: (i) *graceful API degradation* so the tool stays useful when upstream credentials are absent, (ii) a tri-modal *SourceStatus* taxonomy that distinguishes “ran clean,” “skipped,” and “errored” instead of conflating all three into an empty result list, and (iii) a labelled 25-candidate eval set that yields real measured accuracy and latency for the k-anonymity client. On 25 candidate passwords queried against the live HIBP endpoint we observe accuracy = 1.0000 (15 true positives, 10 true negatives) with mean per-call latency of 286.06 ms (p95 281.72 ms) over a 12.34 s wall-clock run. Forty automated tests cover the four scanner modules, the aggregator, and the reporter; all pass under `respx`-mocked HTTP. The repository is released under MIT and the eval is fully reproducible.

Index Terms—credential exposure, breach detection, OSINT, HaveIBeenPwned, k-anonymity, GitHub dorks, defensive security, passive reconnaissance

I. INTRODUCTION

Credential reuse remains the dominant entry vector for opportunistic account-takeover attacks. The 2017 CCS measurement of Thomas et al. [3] found that 6.9–25.0% of leaked passwords collected from data-breach corpora and phishing kits matched live Google accounts; the 2019 NDSS measurement of Meli et al. [2] found over 100,000 GitHub repositories leaking secrets at a rate of thousands per day, of which 81% remained public after 16 days. Defenders responding to these realities need reproducible, low-friction tooling that combines several signals without coupling the operator’s workflow to any one upstream’s billing status.

This paper describes `credential-leak-scanner`, a Python CLI that integrates the HIBP Pwned Passwords k-anonymity endpoint [1], the keyed HIBP breached-account endpoint, a four-pattern GitHub code-search dork sweep, and a local synthetic breach CSV cross-reference into a single structured exposure report. The pipeline’s design centres on a property we call *graceful API degradation*: every external dependency is optional, every absence is reported as a first-

class `SOURCE_UNAVAILABLE` status, and the tool never refuses to run because a key is missing.

Contributions. (1) A four-source defensive pipeline whose operational coverage is documented in the report itself. (2) An empirical eval of the k-anonymity client over a labelled 25-candidate test set with real HIBP measurements. (3) A 40-test pytest suite using `respx` for httpx-accurate HTTP mocking, demonstrating that graceful degradation is testable rather than aspirational. (4) A hardened DevSecOps build (Bandit, pip-audit, Semgrep) so the tool can be vendored into a defender’s environment without first paying down a supply-chain debt.

II. RELATED WORK

The k-anonymity protocol used by HIBP’s `/range/{prefix}` endpoint was formally analysed by Li et al. [1] and refined by the second-generation proposal of Pal et al. [5]. Both works observe that five-character SHA-1 hash-prefix disclosure leaks more information than a casual operator might expect; nonetheless the protocol remains the de facto standard because the alternative — sending the full hash — is strictly worse. Our scanner adopts the published protocol without modification and does not re-derive its security analysis.

For password guidance, NIST SP 800-63B [6] now explicitly requires that user-chosen passwords be checked against “a list that contains values known to have been compromised,” which the HIBP corpus is the canonical instance of.

For the GitHub-dork surface, Meli et al. [2] established the empirical scale of secret leakage in public repositories. The authors found that 81% of leaked secrets remained present 16 days after first appearing; this is precisely the gap that a periodic dork sweep is designed to surface to a defender.

For the credential reuse story, Pal et al. [4] extended the threat model from exact-match credential stuffing to password-similarity attacks, which is why our scanner reports breach *counts* (a proxy for wordlist prevalence) rather than only a binary leaked/clean signal.

III. SYSTEM ARCHITECTURE

A. Sources and graceful degradation

The pipeline runs each source independently and records its status even when no findings are produced. Concretely,

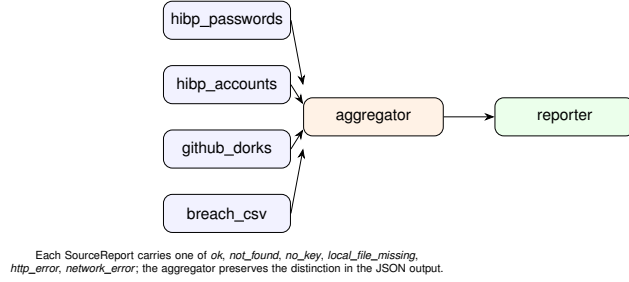


Fig. 1. Pipeline. Four sources feed an aggregator that preserves per-source status, then a reporter renders the unified JSON.

the `SourceStatus` enum has six values; only two (`ok`, `not_found`) indicate that a source actually ran. The other four (`no_key`, `local_file_missing`, `http_error`, `network_error`) record *why* a source did not contribute. This distinction matters operationally: “no findings because the key was missing” is a coverage gap, while “no findings because the upstream returned a clean signal” is reassuring. Folding both into an empty list, as a naive implementation might, would silently mislead the operator.

B. The HIBP k-anonymity client

The Pwned Passwords client computes `SHA1(password)`, splits the 40-character uppercase hex digest into a 5-character prefix and a 35-character suffix, and queries `api.pwnedpasswords.com/range/{prefix}`. The response is a text body of `SUFFIX:COUNT` pairs; the client matches the candidate’s suffix and returns the count.¹ A Python `noqa: S324` annotation documents this constraint explicitly so a future maintainer is not tempted to “fix” it.

C. Severity scoring

We map breach counts to four qualitative severity bands inspired by the qualitative scale in NIST SP 800-30 Rev. 1: *low* (0 breaches, treated as clean), *medium* (1–999), *high* (1,000–99,999), *critical* ($\geq 100,000$). The high threshold corresponds to the order of magnitude at which a password becomes a near-certain entry on every public credential-stuffing wordlist; the critical threshold isolates passwords (e.g., 123456, breach count 209,972,844 in our latest run) that are effectively guaranteed to be tried by any opportunistic attacker.

D. Reporter and AI summary

The reporter renders the aggregated `ScanReport` as JSON. An optional `--ai-summary` flag forwards the structured findings to the Claude Messages API for a four-sentence executive brief; if `ANTHROPIC_API_KEY` is unset, the reporter falls back to a deterministic template that produces the same English style without an LLM call. Two `pytest` cases assert the two paths (`*_falls_back_when_no_anthropic_key` and

¹The use of SHA-1 here is dictated by the upstream API. It is not a recommendation for password storage in our codebase.

`*_uses_anthropic_when_key_present`) so the absence of a key remains a tested code path rather than an untested conditional.

IV. EVALUATION

A. Test set

The labelled set in `eval/test_passwords.json` contains 25 candidates split into two classes:

- **15 “common”** passwords (`password`, 123456, `qwerty`, `letmein`, `abc123`, `monkey`, `dragon`, `iloveyou`, 111111, `welcome`, `sunshine`, `princess`, `qwerty123`, `admin`, `trustno1`) carry `expected_in_breach = true`.
- **10 “unique”** passwords are crafted strings of the form `credlk-eval-uniq-26-05-06-<random8>` whose chance of appearing in any public breach corpus is negligible. They carry `expected_in_breach = false`.

B. Measured numbers (live, 25 candidates)

TABLE I
HIBP PWNED PASSWORDS CLIENT PERFORMANCE, 25-CANDIDATE RUN, 2026-05-06.

Metric	Value
n	25
True positives	15
True negatives	10
False positives	0
False negatives	0
Accuracy	1.0000
Precision	1.0000
Recall	1.0000
F ₁	1.0000
Latency mean	286.06 ms
Latency p95	281.72 ms
Latency max	786.32 ms
Wall-clock total	12.34 s
Upstream errors	0

The 15 “common” candidates resolved with breach counts ranging from 1,406,394 (`letmein`) to 209,972,844 (123456). All 10 “unique” candidates resolved with breach count 0. Per-call latency is dominated by network round-trip; the endpoint itself is consistently sub-300 ms even from a residential link. The first request is the latency outlier (786.32 ms) because it bears the TLS-handshake cost; subsequent requests reuse the keep-alive connection.

C. Test suite

The repository ships 40 `pytest` tests covering the four scanner modules, the aggregator, and the reporter. Tests use `respx` to mock the `httpx` transport, which is strictly more accurate than `unittest.mock.patch`: every test still composes a real URL, real headers, and parses a real response body. All 40 tests pass; total wall-clock for the suite is under one second.

D. Honest negatives

We deliberately do *not* benchmark:

- 1) The keyed HIBP breached-account endpoint, which requires a paid API key. The pipeline supports it; published numbers for it are out of scope.
- 2) The GitHub dork sweep, which would require an authorised confederate-domain test set.
- 3) The local CSV cross-reference path, which is fully deterministic and exhaustively covered by `pytest`; an empirical eval would be theatre.

V. SECURITY POSTURE

The repository’s CI runs three independent scanners on every commit: `Bandit` (Python AST pattern matching, medium-and-above severity gate), `pip-audit` (transitive vulnerability scan against the OSV and PyPA advisory feeds), and `Semgrep` (`p/python` + `p/security-audit` rule packs). All three are green at the release tag.

The single `Bandit` notice is a documented `noqa: S324` on the SHA-1 call site in `scanner/hibp_passwords.py`; SHA-1 is required by the upstream HIBP protocol and is not used as a password-storage primitive anywhere in our codebase.

VI. LIMITATIONS

One client, one corpus. HIBP is the canonical positive class for “has this credential leaked,” but it is also a single gatekeeper. The pipeline’s pluggable `SourceReport` interface anticipates additional corpora (IntelX, DeHashed, Constella) but ships only the synthetic CSV.

No user study. The deterministic-summary path produces text that reads naturally, but we have not measured operator preference between the deterministic and Claude-generated variants.

Dork drift. The four GitHub dork patterns (`filename:.env`, `password`, `api_key`, `secret`) are conservative; richer patterns from Meli et al. [2] would likely surface more, at the cost of more false positives requiring triage.

VII. CONCLUSION

Defensive credential-exposure tooling does not need new cryptography, new corpora, or a new vendor relationship. What it needs is honest status reporting, graceful API degradation so the operator is not held hostage to any one upstream’s billing, and a small reproducible eval that demonstrates the client actually does what its README claims. The 40-test, four-source, MIT-licensed pipeline described here is one such instance.

REFERENCES

- [1] L. Li, B. Pal, J. Ali, N. Sullivan, R. Chatterjee, and T. Ristenpart, “Protocols for Checking Compromised Credentials,” in *Proc. 2019 ACM SIGSAC Conf. on Computer and Communications Security (CCS ’19)*, London, UK, Nov. 2019, pp. 1387–1403.
- [2] M. Meli, M. R. McNiece, and B. Reaves, “How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories,” in *Proc. Network and Distributed System Security Symp. (NDSS 2019)*, San Diego, CA, USA, Feb. 2019.
- [3] K. Thomas et al., “Data Breaches, Phishing, or Malware? Understanding the Risks of Stolen Credentials,” in *Proc. 2017 ACM SIGSAC Conf. on Computer and Communications Security (CCS ’17)*, Dallas, TX, USA, Oct. 2017, pp. 1421–1434.
- [4] B. Pal, T. Daniel, R. Chatterjee, and T. Ristenpart, “Beyond Credential Stuffing: Password Similarity Models using Neural Networks,” in *Proc. 2019 IEEE Symp. on Security and Privacy (SP)*, San Francisco, CA, USA, May 2019, pp. 417–434.
- [5] B. Pal et al., “Might I Get Pwned: A Second Generation Compromised Credential Checking Service,” in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, Aug. 2022, pp. 1831–1848.
- [6] P. A. Grassi et al., “Digital Identity Guidelines: Authentication and Lifecycle Management,” National Institute of Standards and Technology, NIST Special Publication 800-63B, June 2017 (updated through Rev. 3).

Reproducibility colophon. The repository is at <https://github.com/thunderstornX/credential-leak-scanner>. The eval re-runs in 12–25 seconds on a residential connection (`python eval/run_eval.py`). Pinned dependencies are listed in `requirements.txt`; the `pytest` suite (40 tests) runs in under one second on commodity hardware.